AV-SQL: Decomposing Complex Text-to-SQL Queries with Agentic Views

Minh Tam Pham[1], Trinh Pham[1], Thanh Tam Nguyen[1], Tong Chen[2], Hongzhi Yin[2], Quoc Viet Hung Nguyen[1]
[1]Griffith University (Australia), [2]The University of Queensland (Australia)

ABSTRACT

Text-to-SQL translates natural language questions into executable SQL for a given database, enabling natural language access to structured data. Despite rapid advances driven by large language models (LLMs), existing approaches still struggle in real-world settings where database schemas are large and queries are highly compositional. In such cases, providing the full schema often exceeds the context window, while one-shot generation frequently produces non-executable SQL due to syntax errors and incorrect schema linking. To address these challenges, we introduce AV-SQL, a multi-stage framework that combines staged decomposition with execution-grounded validation. AV-SQL decomposes the task into three specialized stages. The Question Rewriting stage uses a rewriter agent to compress and clarify the input question, optionally incorporating external knowledge. The Agent View Generation stage employs a view generator that operates over schema chunks to produce intermediate Common Table Expressions (CTEs), referred to as agent views, which identify and filter relevant schema elements. The SQL stage consists of three agents: a planner agent, an SQL generator agent, and a revisor agent that collaboratively compose these agent views into the final query. Extensive experiments show that AV-SQL achieves 70.38% execution accuracy on the challenging Spider 2.0 benchmark, outperforming state-of-the-art baselines, while remaining competitive on standard datasets with 85.59% on Spider, 72.16% on BIRD and 63.78% on KaggleDBQA.

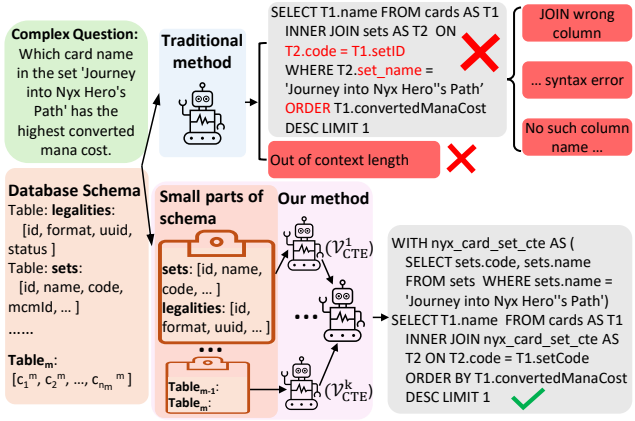


Figure 1: A motivating example from BIRD-dev (sample 501) illustrates the limitations of traditional Text-to-SQL on complex questions over large schemas. Providing the full schema in a single prompt can exceed the context window and increase the risk of syntax and logical errors. Our approach instead splits the schema into smaller chunks, generates and validates agent views (CTE queries: $\mathcal{V}_{\text{CTE}}^{(1)}, \ldots \mathcal{V}_{\text{CTE}}^{(k)}$), and then composes them into the final SQL, improving robustness under long-context constraints.

PVLDB Reference Format:
Minh Tam Pham[1], Trinh Pham[1], Thanh Tam Nguyen[1], Tong Chen[2], Hongzhi Yin[2], Quoc Viet Hung Nguyen[1]. AV-SQL: Decomposing Complex Text-to-SQL Queries with Agentic Views. PVLDB, 20(): XXX-XXX, 2027. doi:XX.XX/XXX.XX

1 INTRODUCTION

Text-to-SQL translates a natural language question into an executable SQL query, enabling natural language interfaces to relational databases. Recent advances in large language models (LLMs) have improved semantic parsing and compositional SQL generation, especially on benchmarks such as Spider-1.0 [53] and BIRD [29]. However, most Text-to-SQL systems [9, 13, 28, 32, 36] implicitly assume that the full database schema can be provided in the prompt. While feasible for small schemas, this becomes impractical for real-world databases with thousands of tables and columns due to context-window limits, degraded attention in long prompts, and increased risk of schema-linking mistakes. Robustness is another barrier: even strong methods [28, 33, 36, 43, 46] frequently generate SQL that fails to execute because of syntax or invalid references, and repairing a single complex query is difficult when errors are entangled. As illustrated in Fig. 1, these issues are amplified when complex questions meet large schemas.

Spider-2.0 [21] makes these limitations explicit by introducing complex questions over large, intricate schemas that often cannot fit into an LLM's context window. Moreover, many gold queries rely on Common Table Expressions (CTEs) to organize multi-step logic, which improves readability for humans but increases the surface area for syntactic and logical errors in automatic generation. Consequently, one-shot generation with global schema prompting becomes unreliable at real-world scale.

To address these challenges, we propose AV-SQL, a multi-stage framework for robust Text-to-SQL. Inspired by Chain-of-Agents long-context decomposition [54], AV-SQL partitions the full schema into table-centric chunks and processes them sequentially. However, unlike standard long-context summarization, Text-to-SQL requires exact schema grounding, so intermediate representations must preserve table and column names rather than compress them away. To achieve this, AV-SQL treats chunk-level reasoning as schema filtering and represents intermediate decisions as executable CTEs. Specifically, AV-SQL consists of three stages: (1) *Question Rewriting*,

where a rewriter agent reformulates the question and optional external evidence into a concise, explicit form; (2) *Agent View Generation*, where view generator agents examine schema chunks, produces agent views as intermediate CTEs, and validates them through execution with iterative repair; and (3) *SQL Generation*, where planner, SQL generator, and revisor agents compose the validated agent views into the final executable SQL. As shown in Fig. 1, this design reduces long-context pressure, makes schema filtering explicit, and improves executability by validating intermediate CTEs before final synthesis. Motivated by these design advantages, our main contributions are as follows:

- **A novel multi-stage agentic Text-to-SQL framework.** We propose AV-SQL, a three-stage framework in which specialized agents perform question reformulation, CTE-based intermediate agent-view construction, and final SQL synthesis.
- **CTE-based agent views with execution-guided repair.** We introduce agent views as executable CTEs that make intermediate schema-filtering decisions explicit and verifiable, and use CTE-level execution feedback to iteratively repair errors before final SQL synthesis.
- **Strong empirical performance.** Extensive experiments show that AV-SQL outperforms state-of-the-art methods on large and complex real-world benchmarks, particularly Spider 2.0, while remaining competitive on standard benchmarks such as Spider, BIRD, and KaggleDBQA.
- **Zero-shot setting.** AV-SQL performs strongly in a fully zero-shot setting, requiring neither post-training nor in-context exemplars, and achieves strong results on Spider 2.0-Snow, Spider, BIRD, and KaggleDBQA.
- **Open-source release.** We will release our code and experimental artifacts to support reproducibility and facilitate future research.

The rest of the paper is organized as follows: §2 reviews related work; §3 formally defines the problem and describes the overview of the AV-SQL framework; §4 details the design of each agent and the overall pipeline; §5 presents experimental results and analysis; and §6 concludes with a summary.

2 RELATED WORK

Text-to-SQL dataset. Text-to-SQL translates a natural language question into an executable SQL query over a relational database. Early studies mainly considered single-domain benchmarks with a single database, such as Restaurants [34, 44], Academic [26], MAS, IMDB, and YELP [52], as well as Scholar [18] and Advising [11]. Later benchmarks broadened the task scope: WikiSQL [55] introduced large-scale table-based evaluation, Spider [53] focused on cross-domain and multi-table SQL, BIRD [29] targeted large databases across 37 professional domains, and KaggleDBQA [19] benchmarked real-world web databases with domain-specific types and unrestricted questions. Spider 2.0 [21] further moves toward realistic deployment by introducing complex questions over large schemas with thousands of columns, making full-schema prompting increasingly impractical.

Text-to-SQL method. Text-to-SQL modeling has progressed through several stages. Before LLMs, neural semantic parsers relied on structured decoding and explicit schema modeling. Representative methods include IRNet [15], which introduced intermediate representations, RAT-SQL [47], which used relation-aware attention for schema encoding and linking, and RESDSQL [27], which improved robustness through schema pruning. Other approaches, such as SLSQL [22], RASAT [38], SADGA [4], and LGESQL [5], further refined relation-aware and graph-based modeling of schema–question interactions. CatSQL [12] bridged rule-based and neural approaches with a sketch-based framework for more accurate and efficient SQL generation. With the rise of LLMs, early work [31, 40] showed that zero-shot and few-shot prompting can generate competitive SQL without task-specific training. Building on this, methods such as DIN-SQL [36], MAC-SQL [46], C3SQL [9], DAIL-SQL [13], DTS-SQL [37], and ZeroNL2SQL [10] improved performance through task decomposition, better demonstrations, structured prompting, and multi-model pipelines. In parallel, supervised fine-tuning approaches such as CodeS [28] and XiYan-SQL [14] strengthened open-source models with Text-to-SQL instruction tuning, while hybrid and multi-candidate methods such as CHESS [43], MCS-SQL [20], and Super-SQL [24] further improved robustness. More recent systems, including CHASE-SQL [35], Alpha-SQL [25], and Long Context NL2SQL [6], incorporate planning, candidate selection, execution feedback, or long-context reasoning to improve performance on complex schemas.

Although existing methods perform strongly on moderate-scale benchmarks such as Spider and BIRD, they struggle on Spider 2.0, where schemas are much larger and queries are more complex. To address this setting, recent work has adopted more iterative and agentic designs, including ReForCE [8], which combines self-refinement, majority-vote consensus, and execution-guided column exploration; DSR-SQL [16], which introduces dual-state reasoning over context and generation; and AutoLink [50], which incrementally expands the relevant schema through autonomous schema linking. Yet performance on Spider 2.0 remains limited, with ReForCE, DSR-SQL, and AutoLink achieving 35.83%, 35.28%, and 34.92% execution accuracy, respectively. In contrast, AV-SQL achieves 70.38% on Spider 2.0, while remaining competitive on Spider (85.59%), BIRD (72.16%) and KaggleDBQA (63.78%).

Long Context Modeling for LLMs. Large language models (LLMs) have finite context windows, making long inputs such as large documents and database schemas difficult to handle. Common strategies include extending context length (e.g.,from 1024 tokens in GPT-2 [39] to 2048 tokens in GPT-3 [3], and up to 128k tokens in GPT-4 [1] and 400k tokens in GPT-5 [42]), though performance can still degrade due to context rot [17]; retrieval-augmented generation (RAG), which retrieves relevant evidence [23, 30, 48, 51] but may omit necessary context [2]; and agentic frameworks such as Chain-of-Agents (CoA), where multiple agents process context segments and a manager aggregates their outputs [54]. Building on CoA [54], AV-SQL adapts long-context decomposition to Text-to-SQL by splitting large schemas into manageable *chunks*, allowing agents to process table/column subsets sequentially rather than loading the full schema into a single prompt. This makes schema filtering explicit, alleviates full-schema prompting, and improves

robustness by validating intermediate agent views before composing the final SQL query. To the best of our knowledge, AV-SQL is the first framework to explicitly apply the Chain-of-Agents paradigm to Text-to-SQL over large schemas and long-context inputs.

3 PROBLEM FORMULATION AND APPROACH OVERVIEW

3.1 Problem Overview

Given a natural language question Q, a database schema $\mathcal{S}$, and optional external knowledge $\mathcal{K}$, the goal of Text-to-SQL is to generate an executable SQL query $\mathcal{Y}$ to answer the question Q. We define the database as $\mathcal{D} = (\mathcal{T}, C, \mathcal{R})$, where $\mathcal{T} = \{\mathcal{T}_1, \mathcal{T}_2, \ldots, \mathcal{T}_m\}$ is the set of m tables, and each table $\mathcal{T}_i$ is associated with a column set $C_i = \{c_1^i, c_2^i, \ldots, c_{n_i}^i\}$, with n_i denoting the number of columns in $\mathcal{T}_i$. Let $C = \{C_1, C_2, \ldots, C_m\}$ be the collection of column sets for all tables, and let $\mathcal{R}$ denote the foreign-key relations among tables. Then, the Text-to-SQL task can be formulated as

$$\mathcal{Y} = f(Q, \mathcal{S}, \mathcal{K} \mid \theta),$$

where the function $f(\cdot \mid \theta)$ can represent a model or pipeline system with the parameter θ, $\mathcal{Y}$ is SQL query can execute against the database instance $\mathcal{D}$ to get the data which use requires in Q

3.2 Challenges

Text-to-SQL systems in practical environments faces several interrelated challenges:

- C1 **Long-context overload.** Real-world databases may be accompanied by long external documents and large schemas with hundreds of tables and thousands of columns, making the input extremely long. To answer a single question, the system may need to process both the external knowledge and the full database schema, which can easily exceed the context window of LLMs. Even when all inputs fit, the long context can dilute attention, making it harder to identify relevant evidence and correctly ground schema elements. As a result, schema grounding weakens, increasing the risk of incomplete or incorrect SQL generation.
- C2 **Limitations of schema reduction.** A common mitigation is to filter the schema by selecting tables and columns relevant to the question. However, in long-context settings, such filtering can reduce accuracy by removing useful schema elements, breaking valid join paths, or providing insufficient structure for correct SQL synthesis.
- C3 **SQL complexity and error propagation.** Real-world questions often require complex SQL with multiple joins, nested conditions, aggregations, and multi-step reasoning. This complexity increases the likelihood of both schema-linking errors and semantic mistakes, and small errors in one part of the query can propagate to the final result.
- C4 **Difficulty of debugging monolithic SQL.** When an LLM generates a single large SQL query, debugging is difficult: if execution fails or results are incorrect, it is often unclear which clause (e.g., join, filter, grouping) caused the issue. This makes iterative correction inefficient and unreliable.

Method	Single candidate	Zero-shot	Handle large DB
DIN-SQL [36], DAIL-SQL [13] , MAC-SQL [46]	✓	✗	✗
MCS-SQL [20], CHESS [43]	✗	✗	✗
Alpha-SQL [25]	✗	✓	✗
ReForCE [8], AutoLink [50]	✗	✓	✓
AV-SQL	✓	✓	✓

Table 1: Comparison of AV-SQL with representative Text-to-SQL methods in terms of (i) single-candidate generation, (ii) zero-shot applicability, and (iii) ability to handle large databases (e.g., Spider 2.0 schemas).

3.3 Framework Overview

To address the challenges above, we propose AV-SQL, a multi-stage Text-to-SQL framework that uses executable agent views, implemented as Common Table Expressions (CTEs), as intermediate representations. Instead of generating a single monolithic SQL query from the full schema, AV-SQL constructs intermediate agent views that make schema filtering explicit, can be executed and validated against the database, and support early error detection before final SQL synthesis. This design reduces full-schema prompting, improves error localization, and makes complex reasoning more manageable.

Table 1 compares AV-SQL with representative Text-to-SQL methods along three practical dimensions: single-candidate generation, zero-shot applicability, and support for large databases. Prompting-based methods such as DIN-SQL, DAIL-SQL, and MAC-SQL typically generate a single candidate, but are not designed for large-schema settings. Methods such as CHESS and MCS-SQL rely on selecting from multiple candidates, while recent Spider 2.0-oriented systems such as ReForCE and AutoLink handle large schemas but also depend on multi-candidate exploration. In contrast, AV-SQL supports single-candidate generation in a fully zero-shot setting while explicitly targeting large-database scenarios. This combination is particularly appealing for practical deployments, where minimizing sampling/selection overhead is important, yet robust handling of long schemas and complex joins remains essential.

In more detail, AV-SQL decomposes Text-to-SQL into three stages that progressively clarify the user intent, construct validated intermediate agent views from schema chunks, and synthesize the final executable SQL. As shown in Fig. 2, the framework consists of the following stages:

- **Question Rewriting**: This stage rewrites the original question, optionally using relevant external knowledge, into a clearer and more explicit form for downstream parsing. It filters useful information from external knowledge, resolves underspecified references, and makes implicit constraints explicit, which reduces linguistic ambiguity and improves schema linking in later stages. For example, it can clarify implicit filters, and specify missing aggregation intent or grouping criteria. The output is a clarified question that preserves the user's intent while making downstream reasoning more reliable. This stage helps mitigate challenges C1 and C3 by focusing attention on relevant information and reducing the likelihood of errors in SQL generation.
- **Agent View Generation**: This stage consists of multiple view-generation agents. Each agent takes a partial schema

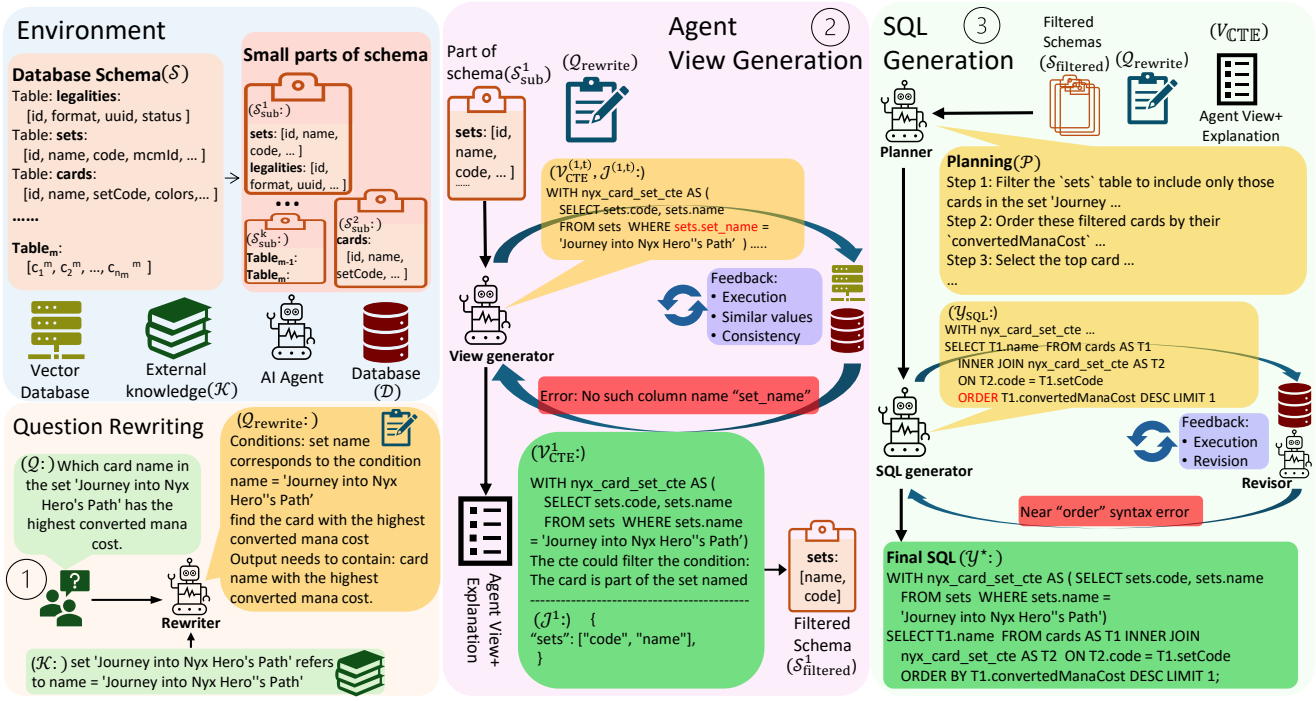


Figure 2: Overview of AV-SQL. The framework consists of three stages: (1) Question Rewriting which reformulates the input question, optionally using external knowledge, into a clearer and more explicit form; (2) Agent View Generation, which decomposes the long-context schema into smaller chunks and produces execution-validated intermediate CTE-based agent views that capture key reasoning steps while identifying relevant schema elements; and (3) SQL Generation, which uses three agents: a planner, a SQL generator, and a revision agent to produce the final executable SQL query from the filtered schema and validated agent views.

context as input and generates a sequence of CTE-based intermediate agent views that incrementally capture the information needed to answer the clarified question. Each agent view is an executable Common Table Expressions (CTE) representing a specific reasoning step, such as identifying candidate entities, constructing join paths, applying key filters, or computing intermediate aggregates. The proposed agent views are validated through execution, or other database-level checks when available, to ensure that the intermediate results are syntactically correct, structurally consistent, and semantically aligned with the intended query. They also provide useful insight into the database contents and the expected form of the final result. This stage addresses challenges C1, C2, C3, and C4 by breaking the full long-context database schema into smaller chunks, verifiable steps, making schema filtering explicit, and enabling early error detection before final SQL synthesis.

- **SQL Generation**: This stage generates the final SQL query. It consists of three agents: a *planner* agent that determines how to construct the query, including the selection target, filtering conditions, required tables for the `FROM` and `JOIN` clauses, and any nested structures; a *SQL generator* agent that produces the query; and a *revision* agent that checks the generated SQL for correctness and consistency with the question. Rather than using the full raw schema, these agents use only on the filtered schema elements and agent views generated in the previous stage, which narrows the search space and reduces the risk of hallucinated columns or incorrect joins. The final SQL is built on the intermediate CTE-based agent views, making the reasoning process more transparent and modular. As a result, this stage helps address challenges C3 and C4 by improving robustness and increasing the likelihood that the generated SQL is both executable and semantically correct.

4 METHODOLOGY

This section presents AV-SQL in detail, as illustrated in Fig. 2. To address large-scale Text-to-SQL over real-world databases with hundreds of tables, our framework organizes the task into three stages: Question Rewriting, Agent View Generation, and SQL Generation. This decomposition reduces schema complexity and improves generation accuracy by dividing the long schema context into smaller and more manageable schema chunks. In particular, Question Rewriting clarifies the input question and extracts useful context, Agent View Generation builds validated intermediate agent views from partial schema contexts, and SQL Generation constructs the final SQL query using the filtered schema and validated CTE-based agent views.

4.1 Database Preprocessing

Before introducing the main stages of AV-SQL, we first apply a set of preprocessing steps to prepare the database. In particular, the preprocessing procedure consists of three components: database schema compression, schema splitting into smaller chunks, and database value preprocessing.

4.1.1 Database schema compression. This step reduces the token length of Spider 2.0 schemas by grouping near-duplicate tables and columns into pattern-based clusters. Spider 2.0 often contains very large schemas with many similar tables and columns whose names differ only by numeric suffixes, such as year, month, or index. Directly serializing these schemas into prompts introduces substantial redundancy and may cause the model to miss important schema details. To reduce this redundancy, we compress the schema by clustering redundant tables and columns based on shared naming patterns. Extending [8], we further verify structural consistency before merging: tables are merged only when they share the same column sets and key information, and columns are grouped only when their types and metadata are consistent. This process compresses large schemas by about 10× (see Fig. 3), substantially reducing prompt length and improving efficiency.

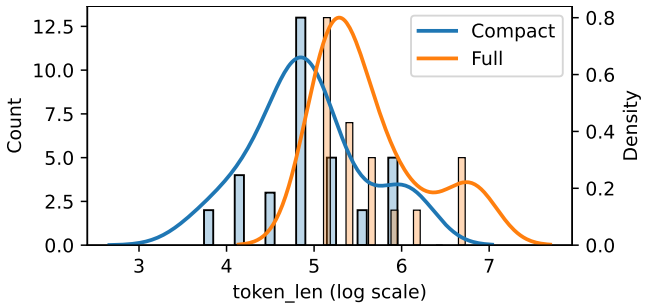


Figure 3: Token-length distribution of large database schemas before and after schema compression.

Example 4.1. In the G360 database in Spider2-snow, the schema contains multiple tables such as
GOOGLE_ANALYTICS_SAMPLE.GA_SESSIONS_20170201,
GOOGLE_ANALYTICS_SAMPLE.GA_SESSIONS_20170202, ...,
GOOGLE_ANALYTICS_SAMPLE.GA_SESSIONS_20170228, ...
Since these tables have similar column sets and metadata, we group them into a single table cluster with the pattern
GOOGLE_ANALYTICS_SAMPLE.GA_SESSIONS_{NUM}.

In the WORLD_BANK database in Spider2-snow, the table WORLD_BANK_GLOBAL_POPULATION.POPULATION_BY_COUNTRY contains columns such as year_2003, year_2004, year_2005, ..., year_2022. Because these columns share the same data type (*Number*) and similar metadata, we group them into a single column cluster with the pattern year_{NUM}.'

4.1.2 Database schema splitting. In large-scale benchmarks such as Spider 2.0, database schemas can be extremely long, reaching up to ∼1.5 million tokens, which exceeds the context window of current LLMs. To make schema grounding feasible for view generation stage of our framework, we partition the full schema into multiple smaller chunks, where each chunk contains one or more *complete* table definitions. Let the full database schema be $\mathcal{S}$. We partition it into k chunks:

$$\Pi(\mathcal{S}) = \{\mathcal{S}^1_{\text{sub}}, \mathcal{S}^2_{\text{sub}}, \ldots, \mathcal{S}^k_{\text{sub}}\}.$$

Each chunk $\mathcal{S}^j_{\text{sub}}$ contains a set of tables together with their complete column information and any foreign-key relations visible within that chunk : $\mathcal{S}^j_{\text{sub}} = (\mathcal{T}^j, \mathcal{C}^j, \mathcal{R}^j)$, where $\mathcal{T}^j$ denotes the set of tables, $\mathcal{C}^j$ the set of columns, and $\mathcal{R}^j$ the set of foreign-key relations visible in chunk j.

There are several possible strategies for splitting $\mathcal{S}$: (i) assigning one table to each chunk, (ii) grouping tables by foreign-key connected components, or (iii) length-based partitioning. In practice, however, large schemas in Spider 2.0 do not provide foreign-key information, so partitioning by connected components is not applicable. We therefore use length-based partitioning, in which complete table definitions are packed into chunks such that the estimated token length of each chunk does not exceed a predefined limit B:

$$\forall \mathcal{S}^j_{\text{sub}} \in \Pi(\mathcal{S}), \quad \left|\mathcal{S}^j_{\text{sub}}\right| < B$$

where $|\cdot|$ denotes the estimated token length of the corresponding schema chunk.

4.1.3 Database Value Preprocessing. Accurately retrieving database values that match a user's natural-language intent is important for reliable Text-to-SQL generation [32]. In practice, SQL queries typically reference only a small subset of values from potentially large databases, most often in filtering predicates such as WHERE clauses. A common challenge is the mismatch between how users mention an entity and how the corresponding value is stored in the database (e.g., a user writes "USA" while the database stores "US"). Following [25, 43], we adopt a two-stage pipeline with offline pre-processing and online retrieval to mitigate this discrepancy.

Offline Pre-processing. In the offline stage, we index text-based entity values that are likely to vary in surface form, such as country names, organization names, abbreviations, and short categorical strings. We exclude primarily numeric values, such as phone numbers, dates, IDs, and postcodes. Each retained value is converted into a MinHash-based LSH signature [7] and stored in a vector database for efficient retrieval.

Online Retrieval. At query time, we extract candidate literals from the generated agent views. For each value, we compute its MinHash LSH signature [7] and retrieve similar entries from the offline index. We then filter the candidates using edit-distance and semantic-similarity thresholds (τ_{edit}, $\tau_{semantic}$), where semantic similarity is computed with *all-MiniLM-L6-v2* [41, 49]. The selected database values are then incorporated into the agent prompt, providing additional context for view generation and SQL synthesis.

4.2 Question Rewriting

The **Question Rewriting** stage employs a single agent, $LLM_{Rewriter}$, to reformulate the user request into a clearer and more explicit form for downstream agents (e.g., the view generator, planner, and SQL generator). It takes as input the original natural-language question Q and optional external knowledge $\mathcal{K}$, and produces a rewritten question Q_{rewrite} that explicitly states the user intent, key constraints, and necessary domain assumptions.

External knowledge $\mathcal{K}$ provides schema-external information such as concept-to-column mappings, metric definitions (e.g., `Average_score`), business rules, and explanations of coded values. In benchmarks like BIRD, this knowledge is typically brief (often just 1–2 sentences). However, in large-scale datasets like Spider 2.0, $\mathcal{K}$ frequently consists of lengthy documents filled with noisy or question-irrelevant details. Providing the full $\mathcal{K}$ directly to downstream LLM agents increases prompt length and can distract the model, degrading Text-to-SQL reliability. Therefore, $LLM_{Rewriter}$ filters out irrelevant information from $\mathcal{K}$, integrates only the pertinent details, and rewrites the original question. The rewritten question structures the key requirements explicitly, for example:

$Q_{\text{rewrite}} = \langle \text{task, required outputs, filters/constraints, sorting/limit} \rangle$.

Formally, the overall rewriting objective is:

$$\begin{aligned} Q_{\text{rewrite}} &= LLM_{Rewriter}(Q, \mathcal{K} \mid Q), \\ \text{s.t.} \quad &\textsc{Intent}(Q_{\text{rewrite}}) = \textsc{Intent}(Q, \mathcal{K}), \end{aligned} \tag{1}$$

Where $\textsc{Intent}(\cdot)$ is denoted as the expressed user intent in the question. As a result, downstream agents receive only the clarified Q_{rewrite} rather than the raw question paired with the full $\mathcal{K}$, which reduces irrelevant context and improves robustness in subsequent reasoning and SQL generation.

4.3 Agent View Generation

The **Agent View Generation** stage is the core component for schema exploration and intermediate reasoning. It consists of multiple view-generator agents, denoted LLM_{View}, each receiving a partial schema chunk. Specifically, each agent takes as input (i) the rewritten question Q_{rewrite} from §4.2 and (ii) a schema chunk $\mathcal{S}^j_{\text{sub}}$ from the schema-splitting procedure in §4.1.2. Restricting each agent to a partial schema reduces prompt length and encourages the model to focus on a manageable subset of tables and columns, mitigating information overload in long-context settings. Each view-generator agent, denoted LLM^j_{View}, produces two outputs: (a) a CTE-based agent view $\mathcal{V}^j_{\text{CTE}}$, consisting of one or more intermediate CTE queries with their natural-language rationale, and (b) a JSON schema selection $\mathcal{J}^j$ identifying the candidate tables and columns needed to answer the question (see Fig. 2 for an example). The generation process for each schema chunk j is defined as:

$$\mathcal{V}^j_{\text{CTE}},\ \mathcal{J}^j = LLM^j_{View}\left(Q_{\text{rewrite}}, \mathcal{S}^j_{\text{sub}}\right), \tag{2}$$

Each CTE query within $\mathcal{V}^{(j)}_{\text{CTE}}$ acts both as a data exploration step over the database and as a compressed logical representation of the reasoning behind table and column selection, encoding operations such as joins, filters, and aggregations in a modular form that can be directly reused during final SQL generation. The JSON selection is defined as $\mathcal{J}^{(j)} = (\mathcal{T}^{(j)}_{\mathcal{J}}, \mathcal{C}^{(j)}_{\mathcal{J}})$, where $\mathcal{T}^j_{\mathcal{J}}$ and $\mathcal{C}^j_{\mathcal{J}}$ denote the selected tables and columns, respectively. These dual outputs serve two purposes: (i) it enables schema filtering for downstream SQL generation, and (ii) supporting automatic grounding quality checks by comparing the JSON selection against the schema elements actually used in the generated CTE.

To improve output quality, an iterative validation and repair procedure is applied to each agent view, covering keyword value extraction and retrieval, execution-based validation to fix syntax and schema-linking errors, and consistency checking between $\mathcal{V}^{(j)}_{\text{CTE}}$ and $\mathcal{J}^{(j)}$. Outputs are refined until valid or a maximum number of repair iterations is reached. Once all schema chunks are processed, the validated agent views and JSON selections are aggregated into a set of executable agent views and a global schema selection, passed to the downstream SQL Generation stage as compact, reusable intermediate reasoning context. The full procedure is described in algorithm 1, with each step detailed below.

Keyword Value Extraction and Retrieval. In addition to producing intermediate queries, the CTE program exposes useful literals (e.g., string constants in `WHERE` clauses). Unlike prior work [25, 43] that extracts keywords directly via prompting, we use `sqlglot` to extract literals from the generated CTE:

$$\mathcal{L}^j_{\text{CTE}} = \textsc{Literal}\left(\mathcal{V}^j_{\text{CTE}}\right). \tag{3}$$

For each $\ell \in \mathcal{L}^j_{\text{CTE}}$, we retrieve and normalize candidate database values denoted $\mathcal{L}^{(j,t)}_{\mathcal{D}}$ using the online value-matching procedure described in §4.1.3. The retrieved candidate set $\mathcal{L}^{(j,t)}_{\mathcal{D}}$ is then feedback to the CTE Agent to regenerate or refine $\mathcal{V}^j_{\text{cte}}$, improving value grounding and increasing the likelihood that the CTE-based agent view executes correctly.

Execution-based validation. This step is to fix any syntax error in CTE query. Each agent view contain one or more CTE queries, which is executable and can be validated by running it against the database. We use `sqlglot` to parse the agent view and extract the underlying SQL for execution, which detects both syntax errors and schema-linking errors. If execution fails, the error message is fed back to the model , which is prompted to revise the agent view. We denote $\textsc{Valid}(\textsc{Exec}(\mathcal{V}^j_{\text{CTE}}, \mathcal{D}))$ as the execution-based validation function, which returns `True` if $\mathcal{V}^j_{\text{CTE}}$ executes successfully on database instance $\mathcal{D}$ without error, and `False` otherwise. If have any error (denoted err^j), this error message will be feedback to the model to repair the CTE query. If $\textsc{Valid}(\textsc{Exec}(\mathcal{V}^j_{\text{CTE}}, \mathcal{D})) =$ `True` mean that there is no error while executing the CTE query $\text{err}^j = \varnothing$

Consistency checking. We also enforce consistency between the CTE-based agent view and the JSON selection, denoted $\textsc{Consistent}(\mathcal{V}^j_{\text{CTE}}, \mathcal{J}^j)$. Let $\mathcal{T}^j_{\mathcal{V}}$ and $\mathcal{C}^j_{\mathcal{V}}$ be the sets of tables and columns used by agent view CTE query $\mathcal{V}^j_{\text{CTE}}$, and JSON selection $\mathcal{J}^j = (\mathcal{T}^j_{\mathcal{J}}, \mathcal{C}^j_{\mathcal{J}})$. The consistency checking require:

- If the JSON output selects no columns and table, the agent must not generate any CTE. It refer that this schema chunk is not relevant to answer the question, so there are not any table or column related to the question.
  $\mathcal{T}^j_{\mathcal{J}} = \varnothing \wedge \mathcal{C}^j_{\mathcal{J}} = \varnothing \implies \mathcal{V}^j_{\text{CTE}} = \varnothing$.
- If the agent generates a CTE, the JSON output must include at least one selected column. It refer that this provided schem chunk related to the question, so JSON selection must choose at least one column or table which relevant to the question. $\mathcal{V}^j_{\text{CTE}} \neq \varnothing \implies \mathcal{C}^j_{\mathcal{J}} \neq \varnothing \vee \mathcal{C}^j_{\mathcal{J}} = \varnothing$
- All tables and columns referenced in the CTE must appear in the JSON selection. This enforces that the CTE-based agent view is use just the table and column which is selected in JSON selection. $(\mathcal{T}^j_{\mathcal{V}} \subseteq \mathcal{T}^j_{\mathcal{J}}) \wedge (\mathcal{C}^j_{\mathcal{V}} \subseteq \mathcal{C}^j_{\mathcal{J}})$

After checking the consistency, the step will also feedback a consistency message denoted by $\texttt{msg}^{(j,t)}_{\text{consistency}}$ which could contain information about which table or column in CTE query is not selected in JSON selection, or any other inconsistency between CTE query and JSON selection. This message will be used to repair both CTE query and JSON selection in the next iteration. This consistency checking step ensure that the response of model is correct in logic. Since the model is required to express schema grounding in two forms–the CTE-based agent view and the JSON selection–any inconsistency between them may indicate a semantic error. Note that we allow redundancy: $\mathcal{J}^j$ may include additional tables/columns not used in the current CTE, since they can be useful when integrating evidence from other schema segments.

Iterative validation and repair. The initial outputs $(\mathcal{V}^{(j,0)}_{\text{CTE}}, \mathcal{J}^{(j,0)})$ may contain errors, including SQL syntax errors, schema-linking errors such as invalid table or column names, and inconsistencies between the CTE and the JSON selection. We therefore apply an iterative validation-and-repair loop, regenerating both outputs until the CTE is executable and the selection is consistent:

$$\mathcal{V}^{(j,t+1)}_{\text{CTE}}, \mathcal{J}^{(j,t+1)} = LLM^j_{View}(\mathcal{Q}_{\text{rewrite}}, \mathcal{S}^j_{\text{sub}}, \mathcal{V}^{(j,t)}_{\text{CTE}}, \mathcal{J}^{(j,t)}, \mathcal{L}^j_{\mathcal{D}}, \text{err}^{(j,t)}, \texttt{msg}^{(j,t)}_{\text{consistency}}) \tag{4}$$

This loop continues until the validation conditions are satisfied or the maximum number of repair iterations denoted $T_{\max}$ is reached.

Aggregation. After all schema chunks have been processed, we aggregate the validated agent views and JSON selections:

$$\mathbb{V}_{\text{CTE}} = \left\{ \mathcal{V}^j_{\text{CTE}} \;\middle|\; \textsc{Valid}\Big(\textsc{Exec}(\mathcal{V}^j_{\text{CTE}}, \mathcal{D})\Big) \wedge \textsc{Consistent}(\mathcal{V}^j_{\text{CTE}}, \mathcal{J}^j) \right\}_{j=1}^{k},$$
$$\mathbb{J} = \left\{ \mathcal{J}^j \;\middle|\; \textsc{Consistent}(\mathcal{V}^j_{\text{CTE}}, \mathcal{J}^j) \right\}_{j=1}^{k}. \tag{5}$$

We then compute the globally selected tables and columns by taking the union over all chunks:

$$\mathcal{T}^\star = \bigcup_{j=1}^{k} \mathcal{T}^j_{\mathcal{J}}, \qquad \mathcal{C}^\star = \bigcup_{j=1}^{k} \mathcal{C}^j_{\mathcal{J}}. \tag{6}$$

Since the consistency check enforces $(\mathcal{T}^j_{\mathcal{V}} \subseteq \mathcal{T}^j_{\mathcal{J}})$ and $(\mathcal{C}^j_{\mathcal{V}} \subseteq \mathcal{C}^j_{\mathcal{J}})$, the union of JSON selections forms a safe superset of all schema elements grounded by the validated CTEs. The global schema selection passed to the SQL Generation stage is: $\mathcal{S}_{\text{filtered}} = \{\mathcal{T}^\star, \mathcal{C}^\star\}$.

The following outputs are passed to the downstream SQL Generation stage:

- **Aggregated Validated Agent Views** $\mathbb{V}_{\text{CTE}}$: validated, executable CTE-based agent view programs with their execution results and natural-language rationales explaining how each agent view contributes to the final answer.
- **Global Schema Selection** $\mathcal{S}_{\text{filtered}}$: a reduced schema derived from the aggregated JSON selections, containing only the relevant tables and columns.

Together, these outputs provide the SQL Generation stage with a compact and well-validated context, enabling it to reuse intermediate reasoning steps, avoid distraction from irrelevant schema elements, and leverage rationales to guide final query construction.

Algorithm 1: Agent view Generation Pipeline

Input: Rewritten question $\mathcal{Q}_{\text{rewrite}}$;
full schema $\mathcal{S}$; and database instance $\mathcal{D}$;
Token budget B for schema segments;
Max repair iterations $T_{\max}$;
Value matching thresholds τ_{edit} and $\tau_{semantic}$
Output: Validated CTE set $\mathbb{V}_{\text{CTE}}$; filtered schema $\mathcal{S}_{\text{filtered}}$

$\{\mathcal{S}^1_{\text{sub}}, \ldots, \mathcal{S}^k_{\text{sub}}\} \leftarrow \textsc{SplitSchemaByTokenBudget}(\mathcal{S}, B)$
$\mathbb{V}_{\text{CTE}} \leftarrow \emptyset$; $\mathbb{J} \leftarrow \emptyset$
for $j \leftarrow 1$ **to** k **do**
  `// Initial generation`
  $(\mathcal{V}^{(j,0)}_{\text{CTE}}, \mathcal{J}^{(j,0)}) \leftarrow LLM^j_{View}(\mathcal{Q}_{\text{rewrite}}, \mathcal{S}^j_{\text{sub}})$
  $t \leftarrow 0$; response_valid $\leftarrow$ `False`
  $\mathcal{L}^j_{\mathcal{D}} \leftarrow \emptyset$ `// retrievel candidate values`
  while $t < T_{\max}$ ***and*** *response_valid* = `False` **do**
    `// Keyword Value Extraction and Retrieval.`
    $\mathcal{L}^{(j,t)} \leftarrow \textsc{Literal}(\mathcal{V}^{(j,t)}_{\text{CTE}})$ **foreach** $\ell \in \mathcal{L}^{(j,t)}$ **do**
      $\mathcal{L}^j_{\mathcal{D}} \leftarrow \mathcal{L}^j_{\mathcal{D}} \cup \textsc{Retrieve}(\ell, \tau_{edit}, \tau_{semantic})$
    `// Execution-based validation.`
    $(\mathbf{Z}^{(j,t)}, \text{err}^{(j,t)}) \leftarrow \textsc{Exec}(\mathcal{V}^{(j,t)}_{\text{CTE}}, \mathcal{D})$
    exec_valid $\leftarrow$ $(\text{err}^{(j,t)} = \emptyset)$
    `// Consistency checking`
    is_consistent, $\texttt{msg}^{(j,t)}_{\text{consistency}} \leftarrow \textsc{Consistent}(\mathcal{V}^{(j,t)}_{\text{CTE}}, \mathcal{J}^{(j,t)})$
    if exec_valid $\wedge$ is_consistent **then**
      response_valid $\leftarrow$ `True`
    else
      $(\mathcal{V}^{(j,t+1)}_{\text{CTE}}, \mathcal{J}^{(j,t+1)}) \leftarrow LLM_{View}(\mathcal{Q}_{\text{rewrite}}, \mathcal{S}^j_{\text{sub}}, \mathcal{V}^{(j,t)}_{\text{CTE}}, \mathcal{J}^{(j,t)}, \mathcal{L}^j_{\mathcal{D}}, \text{err}^{(j,t)}, \texttt{msg}^{(j,t)}_{\text{consistency}})$
    $t \leftarrow t + 1$
  if *response_valid* = `True` **then**
    $\mathbb{V}_{\text{CTE}} \leftarrow \mathbb{V}_{\text{CTE}} \cup \{\mathcal{V}^j_{\text{CTE}}\}$; $\mathbb{J} \leftarrow \mathbb{J} \cup \{\mathcal{J}^j\}$
`// Aggregation`
$\mathcal{T}^\star \leftarrow \bigcup_{\mathcal{T}^j_{\mathcal{J}} \in \mathbb{J}} \mathcal{T}_{\mathcal{J}}$; $\mathcal{C}^\star \leftarrow \bigcup_{\mathcal{C}^j_{\mathcal{J}} \in \mathbb{J}} \mathcal{C}_{\mathcal{J}}$
$\mathcal{S}_{\text{filtered}} \leftarrow \{\mathcal{T}^\star, \mathcal{C}^\star\}$
return $(\mathbb{V}_{\text{CTE}}, \mathcal{S}_{\text{filtered}})$

4.4 SQL Generation

The **SQL Generation** stage synthesizes the final SQL query that answers the user question. It consists of three agents: a planner agent $LLM_{Planner}$, a SQL generator agent LLM_{SQL}, and a revision agent $LLM_{Revisor}$. This stage receives a compact, structured context from earlier stages: (i) the rewritten question $\mathcal{Q}_{\text{rewrite}}$ from §4.2, (ii) the aggregated validated agent views $\mathbb{V}_{\text{CTE}}$ with their execution results and natural-language rationales from §4.3, and (iii) the global schema selection $\mathcal{S}_{\text{filtered}}$ containing only relevant tables and columns from §4.3. This reduced context keeps the agents focused

on the necessary schema elements and lowers the risk of distraction from irrelevant database structures. The stage follows a three-step procedure: $LLM_{Planner}$ constructs a high-level query plan to reduce structural errors; LLM_{SQL} generates the SQL query with execution feedback; and $LLM_{Revisor}$ verifies and revises the generated SQL for semantic correctness. The full procedure is described in algorithm 2, and each step is detailed below.

Planning. Before writing SQL, $LLM_{Planner}$ constructs a high-level query plan to guide generation and reduce structural errors. The plan $\mathcal{P}$ specifies which CTEs and tables to use, required join paths, hidden formulas, necessary projections, and operations such as filtering, grouping, aggregation, and ordering:

$$\mathcal{P} = LLM_{Planner}(Q_{\text{rewrite}}, \mathbb{V}_{\text{CTE}}, \mathcal{S}_{\text{filtered}}) . \tag{7}$$

Generation with execution feedback. Given the plan and the structured context from earlier stages, LLM_{SQL} generates the final SQL query. Similar to the execution-based validation loop in §4.3, execution feedback is incorporated to iteratively repair syntax and schema-linking errors:

$$\mathcal{Y}_{\text{SQL}}^{t+1} = LLM_{\text{SQL}}\Big(\mathcal{P}, Q_{\text{rewrite}}, \mathbb{V}_{\text{CTE}}, \mathcal{S}_{\text{filtered}}, \mathcal{Y}_{\text{SQL}}^{t}, \text{err}^{(t)}\Big) \tag{8}$$

where $\text{err}^{(t)}$ is the execution error message at iteration t. This loop terminates when the query $\mathcal{Y}_{\text{SQL}}$ executes successfully on the database instance $\mathcal{D}$ without error or when the maximum number of iterations $T_{\max}$ is reached.

Semantic verification and revision. Successful execution does not guarantee semantic correctness. After obtaining an executable query from LLM_{SQL}, $LLM_{Revisor}$ is applied to verify whether the result matches the intent of Q_{rewrite}:

$$\mathcal{Y}_{\text{SQL}}^{\star} = LLM_{Revisor}\Big(Q_{\text{rewrite}}, \mathcal{Y}_{\text{SQL}}^{\star}\Big) . \tag{9}$$

If the revision agent confirms that $\mathcal{Y}_{\text{SQL}}$ is semantically correct, it acts as an identity mapping and returns the same query unchanged. Otherwise, $LLM_{Revisor}$ edits the query according to the corrective feedback and produces a revised query $\mathcal{Y}_{\text{SQL}}^{\star}$. This step reduces semantic errors that may survive execution-based validation.

The SQL Generation stage outputs the final verified query $\mathcal{Y}_{\text{SQL}}^{\star}$ and its execution result as the system answer.

Algorithm 2: SQL Generation Pipeline

Input: Rewritten question Q_{rewrite};
Validated CTE set $\mathbb{V}_{\text{CTE}}$;
Filtered schema $\mathcal{S}_{\text{filtered}}$ and database instance $\mathcal{D}$;
Max execution-repair iterations $T_{\max}$
Output: Final SQL query $\mathcal{Y}_{\text{SQL}}$ and execution result $\mathbf{Z}$

```
// (1) Planning
```
$\mathcal{P} \leftarrow LLM_{\text{Planner}}(Q_{\text{rewrite}}, \mathbb{V}_{\text{CTE}}, \mathcal{S}_{\text{filtered}})$
```
// (2) Initial SQL generation from the plan
```
$\mathcal{Y}_{\text{SQL}}^{(0)} \leftarrow LLM_{\text{SQL}}(\mathcal{P}, Q_{\text{rewrite}}, \mathbb{V}_{\text{CTE}}, \mathcal{S}_{\text{filtered}})$
```
// (3) Execution-based validation
```
$t \leftarrow 0$
while $t < T_{\max}$ **do**
  $(\mathbf{Z}^{(t)}, \text{err}^{(t)}) \leftarrow \textsc{Exec}\Big(\mathcal{Y}_{\text{SQL}}^{(t)}, \mathcal{D}\Big)$
  if $err^{(t)} = \varnothing$ **then**
    $\mathcal{Y}_{\text{SQL}}^{\star} \leftarrow \mathcal{Y}_{\text{SQL}}^{(t)}$
    break
  $\mathcal{Y}_{\text{SQL}}^{(t+1)} \leftarrow LLM_{\text{SQL}}\Big(\mathcal{P}, Q_{\text{rewrite}}, \mathbb{V}_{\text{CTE}}, \mathcal{S}_{\text{filtered}}, \mathcal{Y}_{\text{SQL}}^{(t)}, \text{err}^{(t)}\Big)$
  $t \leftarrow t + 1$
```
// (4) Semantic verification and revision
```
$\mathcal{Y}_{\text{SQL}}^{\star} \leftarrow LLM_{\text{Revisor}}\Big(Q_{\text{rewrite}}, \mathcal{Y}_{\text{SQL}}^{\star}\Big)$
$(\mathbf{Z}, _) \leftarrow \textsc{Exec}(\mathcal{Y}_{\text{SQL}}, \mathcal{D})$
return $(\mathcal{Y}_{\text{SQL}}, \mathbf{Z})$

5 EXPERIMENTS

This section presents experiments evaluating the effectiveness, scalability, and reliability of AV-SQL. To systematically assess the contribution of each design choice, we organize our experiments around the following research questions (RQs).

- RQ1 **(Normal-scale performance):** How well does the framework perform on standard Text-to-SQL benchmarks: Spider, BIRD and KaggleDBQA?
- RQ2 **(Large-scale scalability):** How well does the framework perform on large scale, enterprise oriented benchmarks Spider 2.0-Snow?
- RQ3 **(Multi-Candidate SQL Generation):** What is the performance of AV-SQL under a multiple SQL candidates setting?
- RQ4 **(Ablation study)** *(Component contribution):* How do the key components–the rewriter, planner, revisor, and execution feedback for the view generator and SQL generator–contribute to overall performance? *(View Generation Output Contribution):* How do the two outputs of the Agent View Generation stage, *Aggregated Validated Agent Views* and *Global Schema Selection* contribute to the correctness of the final SQL query?
- RQ5 **(Schema Filtering):** What is the accuracy of the Agent View Generation stage in identifying and filtering relevant tables and columns?
- RQ6 **(Component cost analysis)**:What are the runtime and computational cost contributions of each component in the proposed framework?
- RQ7 **(Error analysis)**: What are the common error types and failure modes of the proposed framework?
- RQ8 **(Hybrid LLM Configuration)**: What is the impact of using heterogeneous LLM backbones across agents, compared to a single shared backbone?

5.1 Experimental settings

Databases and benchmarks. We evaluate our approach on both standard Text-to-SQL benchmarks and a large-scale, enterprise-oriented benchmark.

- Normal-scale databases. We evaluate on three widely used Text-to-SQL benchmarks: Spider [53], which contains 200 multi-table databases across 138 diverse domains; BIRD [29], which includes 95 databases totaling 33.4 GB from 37 professional domains; and KaggleDBQA [19], a cross-domain benchmark of real-world web databases with domain-specific

data types, original formatting, and unrestricted questions. For BIRD, we use the development version *bird_dev_20251106*.

- Large-scale databases. To assess scalability on real-world schema, we evaluate on Spider2-Snow a Spider 2.0 [21] subtask with 547 complex questions designed for enterprise-scale Text-to-SQL. Compared with standard academic benchmarks, it features much larger and more heterogeneous schemas, averaging about 800 columns per database, which makes schema understanding and SQL generation significantly more challenging. The Spider2-Snow requires generating SQL in the Snowflake dialect.

Evaluation metrics. We evaluate model performance using *Execution Accuracy* (EX). EX measures whether the SQL query predicted by the model produces the same execution result as the ground-truth SQL when run on the target database, and we use it as the primary indicator of correctness. It is important to note that the definition of EX differs across benchmarks. BIRD adopts a strict evaluation protocol, requiring the execution results of a generated query to exactly match the gold result in both returned content and column order. In contrast, Spider 2.0-Snow uses a more lenient evaluation script. To account for potential ambiguity in user intent, it only requires that the returned results contain the core information of the gold answer, without constraining column order or penalizing the presence of additional columns, thereby reducing false negatives during evaluation.

Model. Our experiments employ several large language models (LLMs) from closed-source flagship models (Gemini-3-Pro-Preview and GPT-5-Mini) and open-source models (Qwen2.5-32B and LLaMA-3.3-70B). Unless otherwise specified, we set the decoding temperature to 1.0, we use $T_{max} = 5$ turn execution feedback.

Baselines. We compare our approach against representative Text-to-SQL systems, including both existing frameworks and pretrained models such as CodeS [28], DIN-SQL [36], DAIL-SQL [13], CHESS [43], etc .... For Spider 2.0-Snow, we additionally include baselines specifically designed for Spider 2.0 [21], including Spider-Agent [21], AutoLink [50], DSR-SQL [16] and ReFoRCE [8], etc ....

5.2 Normal databases performance (RQ1)

To answer RQ1, AV-SQL is evaluated on three standard Text-to-SQL benchmarks: BIRD, Spider, and KaggleDBQA. Table 2 reports Execution Accuracy (EX) on BIRD and Spider, comparing AV-SQL against both *multi-candidate* and *single-candidate* baselines, along with latency where available. AV-SQL with Gemini-3-Pro achieves the best BIRD result among single-candidate methods (72.16%) and competitive Spider accuracy (85.59%). With Qwen2.5-32B, AV-SQL reaches 67.99% on BIRD, closely approaching Alpha-SQL-32B (69.70%) — a multi-candidate method with latency exceeding 1000s — while avoiding expensive candidate search.

Table 3 and Table 4 break down performance by difficulty level on Spider and BIRD, respectively. On Spider, AV-SQL with Gemini-3-Pro shows strong results on Hard queries (89.08%), surpassing all baselines at that level. On BIRD, AV-SQL with Gemini-3-Pro achieves the highest accuracy across all difficulty tiers, including Simple (76.60%), Moderate (67.46%), and Challenging (60.69%).

Table 5 evaluates AV-SQL on KaggleDBQA, a more challenging out-of-domain benchmark. AV-SQL with Gemini-3-Pro attains 63.78% overall EX, outperforming all listed baselines including LC-NL2SQL (61.10%), and shows strong results on Hard (67.35%) and Extra Hard (56.41%) queries.

Method	Infer model	BIRD↑	Spider↑	Latency(s)↓
Multi-candidate				
MCS-SQL [20]	GPT-4	63.36	86.80	-
Alpha-SQL-7B [25]	Qwen2.5-7B	66.80	84.00	>1000
Alpha-SQL-14B [25]	Qwen2.5-14B	68.70	87.00	>1000
Alpha-SQL-32B [25]	Qwen2.5-32B	69.70	-	>1000
XiyanSQL-32B [14]	finetuned-32B	73.34	-	-
CHESS [43]	GPT-4	68.31	87.2	118.61
ZeroNL2SQL [10]	GPT-4	-	84.00	-
Single-candidate				
DIN-SQL [36]	GPT-4	50.72	82.80	24.09
DAIL-SQL [13]	GPT-4	55.90	83.10	-
MAC-SQL [46]	GPT-4	59.59	**86.75**	24.64
SFT Llama2-13B [45]	finetuned-13B	53.91	81.60	-
CodeS-7B [28]	finetuned-7B	57.17	85.40	1.87
CodeS-15B [28]	finetuned-15B	58.47	84.90	3.52
XiyanSQL-32B [14]	finetuned-32B	66.88	-	-
XiyanSQL-7B [14]	finetuned-7B	60.10	-	-
LC-NL2SQL [6]	Gemini-1.5-pro	67.41	87.10	12.3
AV-SQL				83.38
+	Qwen2.5-32B	67.99	84.33	–
+	LLaMA 3.3-70B	68.19	83.66	–
+	Gemini-3-Pro	**72.16**	85.59	–

Table 2: Execution Accuracy (EX) on the BIRD and Spider benchmark.

Method	Easy	Medium	Hard	Extra
CodeS-7B [28]	94.80	91.00	75.30	66.90
CodeS-15B [28]	95.60	90.40	78.20	61.40
DIN-SQL(GPT-4) [36]	92.30	87.40	76.40	62.70
DAIL-SQL(GPT-4) [13]	91.50	90.10	75.30	62.70
Alpha-SQl-7B [25]	94.00	89.20	76.40	63.30
Alpha-SQl-14B [25]	94.00	91.00	79.90	72.30
AV-SQL				
+Qwen2.5-32B	90.73	90.13	79.31	64.46
+LLaMA 3.3-70B	91.13	85.43	82.76	68.67
+Gemini-3-Pro	90.32	86.55	89.08	72.29

Table 3: Performance on Spider comparison across difference hardness levels.

5.3 Large scale database performance (RQ2)

To address RQ2 under more challenging conditions, we evaluate on Spider 2.0-Snow, which stress-tests text-to-SQL systems with substantially larger schemas and higher domain heterogeneity. Table 6 reports EX and average token usage on Spider 2.0-Snow, a large-scale benchmark where conventional pipelines degrade sharply. Standard methods such as CodeS-15B and DIN-SQL score 0.00%, and even Spider-Agent peaks at 23.58% (o1-preview), while recent

Method	Simple	Mod	Chall
DAIL-SQL(GPT-4) [13]	63.00	45.60	43.10
MAC-SQL(GPT-4) [46]	65.73	52.69	40.28
CodeS-7B [28]	64.60	46.90	40.30
CodeS-15B [28]	65.80	48.80	42.40
Alpha-SQL-7B [25]	72.60	59.30	53.10
Alpha-SQL-14B [25]	74.60	61.00	55.90
Alpha-SQL-32B [25]	74.50	64.00	57.20
AV-SQL			
+Qwen2.5-32B	72.37	64.44	52.41
+LLaMA 3.3-70B	72.91	62.28	57.93
+Gemini-3-Pro	**76.60**	**67.46**	**60.69**

Table 4: Execution accuracy on BIRD dev set by difficulty level. For brevity, the abbreviation "Mod" stands for "Moderate" while "Chall" denotes "Challenging"

Method	EX (%)				
	Easy	Medium	Hard	Extra	All
DIN-SQL(gpt-4) [36]	-	-	-	-	27.00
Incremental ICL [10](gpt-3.5)	-	-	-	-	34.60
RESDSQL [27]	-	-	-	-	31.90
RAT-SQL [47]	-	-	-	-	13.56
ZeroNL2SQL(gpt-4) [10]	-	-	-	-	42.40
ZeroNL2SQL(gpt-3.5) [10]	-	-	-	-	44.90
LC-NL2SQL [6](Gemini-1.5-Pro)	65.94	57.83	64.86	50.81	61.10
AV-SQL					
+LLaMA 3.3-70B	65.96	56.00	57.14	30.77	53.51
+Gemini-3-Pro	72.34	58.00	67.35	56.41	63.78

Table 5: Performance comparison on KaggleDBQA across various models and tasks.

methods ReFoRCE [8] reach only ~ 35%. AV-SQL substantially outperforms all baselines, achieving up to 70.38% EX with Gemini-3-Pro, while consuming fewer tokens (92k) than DAIL-SQL (124k) and ReFoRCE (230k).

Table 7 further breaks down performance by difficulty level. Spider-Agent (o1-preview) degrades sharply across easy, medium, and hard splits (39.84%, 21.14%, 15.61%), whereas AV-SQL with Gemini-3-Pro maintains consistently high accuracy (74.22%, 72.36%, 64.74%). The performance gap widens significantly on harder queries, demonstrating AV-SQL's stronger handling of compositional reasoning challenges in large-schema settings.

5.4 Multi-Candidate SQL Generation

To answer RQ3, AV-SQL is evaluated in a multi-candidate setting on Spider2-Snow, where the SQL generation agent is prompted to output k candidate queries. Performance is measured using *recall* Execution Accuracy, which considers a question correctly answered if at least one of the k generated candidates is executable correct. As shown in Fig. 4, increasing k yields consistent gains for both backbones: Gemini-3-Pro improves from 70.38% at k=1 to 74.77% at k=16, while GPT-5-mini rises from 65.08% to 74.00% over the same range. These results indicate that generating more candidates increases the chance of producing at least one correct SQL query, improving robustness on challenging large-schema queries. This

Method	Infer model	EX↑	Avg Tokens↓
SFT CodeS-15B [28]	CodeS-15B	0.00	–
DIN-SQL [36]	GPT-4o	0.00	32k
CHESS [43]	GPT-4o	1.28	–
DAIL-SQL [13]	GPT-4o	2.20	124k
LC-NL2SQL [6]	gpt-5-mini	0.00	–
Spider-Agent [21]	Qwen2.5-Coder	5.48	–
Spider-Agent [21]	GPT-4o	12.98	–
Spider-Agent [21]	o3-mini	19.20	–
Spider-Agent [21]	o1-preview	23.58	–
ReFoRCE [8]	o1-preview	31.26	230k
ReFoRCE [8]	o3	35.83 (62.89)	–
AutoLink [50]	DeepSeek-R1	34.92 (54.84)	–
DSR-SQL [16]	DeepSeek-R1	35.28 (63.80)	–
AV-SQL			92k
+	Qwen2.5-32B	25.05	–
+	LLama3.3-70B	27.42	–
+	gpt-5-mini	65.08	–
+	Gemini-3-Pro	**70.38**	–

Table 6: Execution Accuracy (EX) on Spider 2.0-Snow. Values in parentheses mark cases where the score reported in the published paper differs substantially from the official leaderboard; the parenthesized number corresponds to the leaderboard score.

Method	Infer model	Easy	Medium	Hard
Spider-Agent [21]				
+	o1-preview	39.84	21.14	15.61
+	o3-mini	31.25	18.29	11.56
+	GPT-4o	24.22	11.38	6.94
+	Qwen2.5-Coder	4.47	2.31	5.48
AV-SQL				
+	Qwen2.5-32B	39.84	24.39	15.03
+	LLama3.3-70B	38.28	32.11	12.71
+	Gpt-5-mini	**75.78**	68.29	52.60
+	Gemini-3-Pro	74.22	**72.36**	**64.74**

Table 7: Execution Accuracy (EX) by difficulty level (easy/medium/hard) on Spider2-snow dataset

also suggests that a effective candidate selector–one that reliably identifies the correct query among candidates–could further boost overall performance.

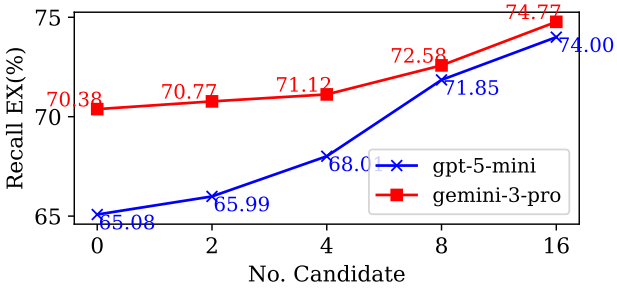


Figure 4: Execution accuracy (EX) recall on Spider 2.0-snow under the multi-candidate setting.

Setting	Spider-2-snow		Spider		BIRD		KaggleDBQA	
	GPT-5 -mini	Gemini -3-Pro	Llama -3.3-70B	Gemini -3-Pro	Llama -3.3-70B	Gemini -3-Pro	Llama -3.3-70B	Gemini -3-Pro
w/o rewriter	57.22	64.90	78.34	84.53	65.45	71.64	48.65	62.16
w/o exe feedback (SQL)	50.82	63.99	80.33	84.91	64.54	71.77	51.35	61.62
w/o exe feedback (CTE+SQL)	41.68	57.77	79.01	84.77	64.47	70.90	50.81	61.08
w/o planner	55.76	63.99	79.40	84.62	65.97	72.23	54.05	61.62
w/o reviseSQL	60.15	65.08	82.50	**87.14**	67.47	**72.82**	**54.59**	**65.41**
All components	**65.08**	**70.38**	**83.66**	85.59	**68.19**	72.16	53.51	63.78

Table 8: Execution accuracy (EX) ablation study of major pipeline components on Spider2-snow, Spider, BIRD dataset.

Information provided	Spider2-snow		Spider		BIRD		KaggleDBQA	
	GPT-5 -mini	Gemini -3-Pro	LLama -3.3-70B	Gemini -3-Pro	LLama -3.3-70B	Gemini -3-Pro	LLama -3.3-70B	Gemini -3-Pro
Only Aggregated Validated Agent Views ($\mathbb{V}_{\text{CTE}}$)	58.68	64.72	79.83	85.42	67.41	71.64	51.89	62.70
Only Global Schema Selection ($\mathcal{S}_{\text{filtered}}$)	57.59	63.25	79.57	84.72	65.71	71.77	50.27	61.62
All information ($\mathbb{V}_{\text{CTE}}$ + $\mathcal{S}_{\text{filtered}}$)	**65.08**	**70.38**	**83.66**	**85.59**	**68.19**	**72.16**	**53.51**	**63.78**

Table 9: Ablation study on which outputs of the CTE generation stage are provided to the SQL generation stage across Spider2-snow, Spider, and BIRD.

5.5 Ablation study

Component contribution. To answer RQ4, a controlled ablation is conducted by disabling one component at a time — the rewriter, planner, revisor, and execution feedback for the view generator and SQL generator — while keeping all others intact. Table 8 shows that the full pipeline consistently outperforms all ablated variants across Spider2.0-Snow, Spider, BIRD, and KaggleDBQA. Removing execution feedback from both CTE and SQL generation causes the largest drops, with Gemini-3-Pro falling from 70.38% to 57.77% and GPT-5-mini from 65.08% to 41.68% on Spider2.0-Snow, confirming that multi-stage correction is critical for preventing error propagation. Removing SQL-only feedback has a smaller but consistent impact (70.38% → 63.99%), while ablating the rewriter, planner, and revisor yields incremental degradation, with larger gaps observed on harder benchmarks. Notably, removing the revisor yields marginal improvements on Spider, BIRD, and KaggleDBQA (e.g., Gemini-3-Pro: 85.59% → 87.14%, 72.16% → 72.82%, 63.78% → 65.41%), suggesting that the revisor primarily benefits complex large-schema queries such as those in Spider2.0-Snow, and may occasionally overcorrect on simpler or out-of-domain benchmarks.

View Generation Output Contribution. To answer the second aspect of RQ4, this experiment isolates the contribution of each output from the View Generation stage to final SQL generation: (i) *Aggregated Validated Agent Views* ($\mathbb{V}_{\text{CTE}}$) and (ii) *Global Schema Selection* ($\mathcal{S}_{\text{filtered}}$) As shown in Table 9, combining both outputs consistently achieves the highest accuracy across all datasets and backbones. On Spider2.0-Snow, the full combination reaches 70.38% (Gemini-3-Pro) and 65.08% (GPT-5-mini), outperforming either output alone. Consistent gains appear on Spider (83.66% vs. 79.83%/79.57%), BIRD (72.16% vs. 71.64%/71.77%), and KaggleDBQA (63.78% vs. 62.70%/61.62%), confirming that schema focusing and executable intermediate reasoning are complementary and jointly necessary for accurate SQL generation.

5.6 Schema Filtering

To answer RQ5, this experiment evaluates schema filtering quality by measuring precision and recall of schema elements selected by each method. Precision measures whether selected elements are correct (few false positives), while recall measures whether all relevant elements are captured (few false negatives). Three baseline strategies are compared: (i) **MACSQL** [46], which prompts the model with the full schema to return relevant tables and columns; (ii) **CHESS** [43], which evaluates table relevance one at a time before selecting columns from relevant tables; and (iii) an **encoder–decoder** schema selector [28], which predicts relevant schema items from the question and schema as input. As shown in Table 10, CHESS achieves high recall (90.58–91.12%) but low precision (27.50–36.87%), indicating excessive false positives. MACSQL achieves a better balance (59.41–70.62% precision, 63.88–86.45% recall), while the encoder–decoder baseline yields 41.50% precision and 79.60% recall. AV-SQL's CTE generation achieves the highest precision across all backbones (81.41–86.34% with proprietary models; 69.74–77.76% with open-weight models) while maintaining competitive recall (81.59–82.70%), confirming that CTE-based intermediate reasoning produces cleaner and more reliable schema filtering than prompting or learned approaches.

Method	Infer model	Precision	Recall
MACSQL [46]			
+	Qwen2.5	62.00	63.88
+	LLaMA 3.3	68.30	68.91
+	GPT-5-Mini	59.41	79.22
+	Gemini-3-Pro	70.62	86.45
CHESS [43]			
+	Qwen2.5	49.03	77.78
+	LLaMA 3.3	27.99	76.70
+	GPT-5-Mini	27.50	91.12
+	Gemini-3-Pro	36.87	90.58
Encoder-Decoder [28]		41.50	79.60
CTE generation in AV-SQL			
+	Qwen2.5	69.74	69.27
+	LLaMA 3.3	77.76	74.67
+	GPT-5-Mini	81.41	**82.70**
+	Gemini-3-Pro	**86.34**	81.59

Table 10: Precision and recall of schema filtering across methods and inference models in Spider2-snow dataset.

5.7 Component cost analysis

To answer RQ6, Fig. 5 presents the token usage and runtime breakdown per pipeline stage for AV-SQL with Gemini-3-Pro on Spider2-Snow. The CTE generation stage dominates both dimensions, consuming 77.23% of total tokens (71.30% input, 5.93% output) and 55.45% of total runtime, reflecting the cost of processing multiple schema chunks with iterative validation. The SQL generation and revisor stages account for 8.04% and 7.03% of token usage, and 17.06% and 15.77% of runtime, respectively. The planner contributes 5.87% of tokens and 7.83% of runtime, while the rewriter is the lightest stage at 1.83% of tokens and 3.89% of runtime. Overall, the cost profile is heavily concentrated in the CTE generation stage, which is expected given its role as the core schema exploration and intermediate reasoning component, while the remaining stages impose relatively modest overhead.

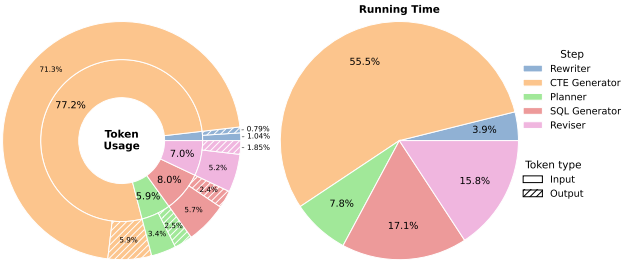


Figure 5: Token usage and running time breakdown by pipeline stage for AV-SQL with Gemini-3-Pro on Spider2-snow. The left nested donut chart shows the proportion of tokens usage, while the right pie chart shows the proportion of total runtime spent on each stage.

5.8 Error Analysis

To answer RQ7, a human evaluation is conducted on the failure cases of AV-SQL with Gemini-3-Pro on Spider2-Snow. Fig. 6 shows the resulting error taxonomy. Filtering (36.23%) and aggregation (34.06%) errors dominate, together accounting for over 70% of failures, indicating that complex filter conditions and aggregation logic over large schemas remain the primary challenges. Schema-linking errors contribute 10.87%, while syntax errors account for only 9.42%, confirming that execution feedback effectively suppresses most malformed queries. The remaining errors correspond to ground truth disagreements (6.52%) and miscellaneous cases (2.90%). These results suggest that future work should prioritize more precise filter and aggregation reasoning.

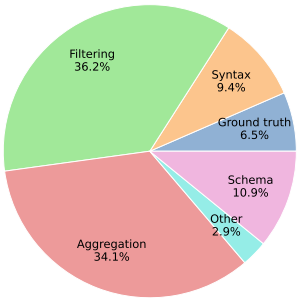


Figure 6: Error taxonomy of AV-SQL with Gemini-3-Pro on Spider2-snow.

5.9 Hybrid LLM configuration

To answer RQ8, Table 11 evaluates the effect of assigning different LLM backbones to different agents, compared to a single unified backbone. Homogeneous configurations with strong LLM backbones achieve the best results, all-GPT-5-mini 65.08%, and all-Gemini-3-Pro 70.38% on Spider2-Snow. Replacing only the SQL generation stage with a stronger model while retaining LLaMA-3.3 for the rewriter and CTE generator improves performance from 27.42% to 40.40%, demonstrating that a stronger SQL generator can better exploit the filtered context produced by the CTE stage. This is also cost-efficient, since the CTE generation stage dominates token usage, meaning the expensive backbone is only applied where it matters most. However, mixing backbones does not consistently improve performance: using Gemini-3-Pro for the rewriter and CTE generator but GPT-5-mini for view generation yields only 61.60%, falling below the homogeneous GPT-5-mini baseline (65.08%), suggesting that misalignment between the CTE generator and SQL generator can degrade performance even when individual components are strong.

Rewriter	View Gen.	SQL Generation			EX↑
		Planner	SQL Gen.	Revisor	
Homogeneous					
LLaMA-3.3	LLaMA-3.3	LLaMA-3.3	LLaMA-3.3	LLaMA-3.3	27.42
GPT-5-mini	GPT-5-mini	GPT-5-mini	GPT-5-mini	GPT-5-mini	65.08
Gemini-3-Pro	Gemini-3-Pro	Gemini-3-Pro	Gemini-3-Pro	Gemini-3-Pro	**70.38**
Hybrid					
LLaMA-3.3	LLaMA-3.3	LLaMA-3.3	GPT-5-mini	GPT-5-mini	40.40
GPT-5-mini	LLaMA-3.3	GPT-5-mini	GPT-5-mini	GPT-5-mini	42.96
Gemini-3-Pro	LLaMA-3.3	Gemini-3-Pro	Gemini-3-Pro	Gemini-3-Pro	55.21
Gemini-3-Pro	GPT-5-mini	Gemini-3-Pro	Gemini-3-Pro	Gemini-3-Pro	61.60

Table 11: Execution Accuracy (EX) on Spider2-Snow under hybrid LLM configurations, where different backbone models are assigned to different agents.

6 CONCLUSIONS

This paper presents AV-SQL, a multi-agent Text-to-SQL framework designed for robust SQL generation across both standard and large-scale database settings. AV-SQL decomposes the generation process into three stages: (1) *Question Rewriting* rewrites the input question (and optional external knowledge) into a clearer and more concise form; (2) *Agent View Generation* processes schema chunks, producing validated CTE-based agent views and a global schema selection; and (3) *SQL Generation* composes the validated intermediate views and filtered schema to generate the final executable SQL query. Extensive experiments on Spider, BIRD, KaggleDBQA, and Spider2.0-Snow demonstrate that AV-SQL achieves competitive or state-of-the-art execution accuracy across all benchmarks, with particularly strong gains on Spider2.0-Snow where large schemas pose significant challenges to conventional pipelines. Collectively, these results indicate that processing schema chunks through the Agent View Generation stage– producing validated intermediate reasoning and filtered schema context–provides a practical and scalable solution for Text-to-SQL generation in real-world large-scale database settings.

REFERENCES

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[2] Scott Barnett, Stefanus Kurniawan, Srikanth Thudumu, Zach Brannelly, and Mohamed Abdelrazek. 2024. Seven failure points when engineering a retrieval augmented generation system. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering-Software Engineering for AI*. 194–199.

[3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *NeurIPS* 33 (2020), 1877–1901.

[4] Ruichu Cai, Jinjie Yuan, Boyan Xu, and Zhifeng Hao. 2021. Sadga: Structure-aware dual graph aggregation network for text-to-sql. *NeurIPS* 34 (2021), 7664–7676.

[5] Ruisheng Cao, Lu Chen, Zhi Chen, Yanbin Zhao, Su Zhu, and Kai Yu. 2021. LGESQL: Line Graph Enhanced Text-to-SQL Model with Mixed Local and Non-Local Relations. In *IJCNLP*. 2541–2555.

[6] Yeounoh Chung, Gaurav T Kakkar, Yu Gan, Brenton Milne, and Fatma Ozcan. 2025. Is long context all you need? leveraging LLM's extended context for NL2SQL. *arXiv preprint arXiv:2501.12372* (2025).

[7] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the twentieth annual symposium on Computational geometry*. 253–262.

[8] Minghang Deng, Ashwin Ramachandran, Canwen Xu, Lanxiang Hu, Zhewei Yao, Anupam Datta, and Hao Zhang. 2025. ReFoRCE: A Text-to-SQL Agent with Self-Refinement, Consensus Enforcement, and Column Exploration. *arXiv preprint arXiv:2502.00675* (2025).

[9] Xuemei Dong, Chao Zhang, Yuhang Ge, Yuren Mao, Yunjun Gao, Jinshu Lin, Dongfang Lou, et al. 2023. C3: Zero-shot text-to-sql with chatgpt. *arXiv preprint arXiv:2307.07306* (2023).

[10] Ju Fan, Zihui Gu, Songyue Zhang, Yuxin Zhang, Zui Chen, Lei Cao, Guoliang Li, Samuel Madden, Xiaoyong Du, and Nan Tang. 2024. Combining small language models and large language models for zero-shot NL2SQL. *Proceedings of the VLDB Endowment* 17, 11 (2024), 2750–2763.

[11] Catherine Finegan-Dollak, Jonathan K Kummerfeld, Li Zhang, Karthik Ramanathan, Sesh Sadasivam, Rui Zhang, and Dragomir Radev. 2018. Improving text-to-sql evaluation methodology. *arXiv preprint arXiv:1806.09029* (2018).

[12] Han Fu, Chang Liu, Bin Wu, Feifei Li, Jian Tan, and Jianling Sun. 2023. Catsql: Towards real world natural language to sql applications. *Proceedings of the VLDB Endowment* 16, 6 (2023), 1534–1547.

[13] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-sql empowered by large language models: A benchmark evaluation. *Proc. VLDB Endow.* 17, 5 (Jan. 2024), 1132–1145.

[14] Yingqi Gao, Yifu Liu, Xiaoxia Li, Xiaorong Shi, Yin Zhu, Yiming Wang, Shiqi Li, Wei Li, Yuntao Hong, Zhiling Luo, et al. 2024. XiYan-SQL: A Multi-Generator Ensemble Framework for Text-to-SQL. *arXiv preprint arXiv:2411.08599* (2024).

[15] Jiaqi Guo, Zecheng Zhan, Yan Gao, Yan Xiao, Jian-Guang Lou, Ting Liu, and Dongmei Zhang. 2019. Towards Complex Text-to-SQL in Cross-Domain Database with Intermediate Representation. In *ACL*. 4524–4535.

[16] Zhifeng Hao, Qibin Song, Ruichu Cai, and Boyan Xu. 2025. Text-to-SQL as Dual-State Reasoning: Integrating Adaptive Context and Progressive Generation. *arXiv preprint arXiv:2511.21402* (2025).

[17] Kelly Hong, Anton Troynikov, and Jeff Huber. 2025. Context rot: How increasing input tokens impacts llm performance. *URL https://research. trychroma. com/context-rot, retrieved October* 20 (2025), 2025.

[18] Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, Jayant Krishnamurthy, and Luke Zettlemoyer. 2017. Learning a neural semantic parser from user feedback. *arXiv preprint arXiv:1704.08760* (2017).

[19] Chia-Hsuan Lee, Oleksandr Polozov, and Matthew Richardson. 2021. KaggleDBQA: Realistic evaluation of text-to-SQL parsers. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. 2261–2273.

[20] Dongjun Lee, Choongwon Park, Jaehyuk Kim, and Heesoo Park. 2025. Mcs-sql: Leveraging multiple prompts and multiple-choice selection for text-to-sql generation. In *Proceedings of the 31st International Conference on Computational Linguistics*. 337–353.

[21] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, et al. 2024. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. *arXiv preprint arXiv:2411.07763* (2024).

[22] Wenqiang Lei, Weixin Wang, Zhixin Ma, Tian Gan, Wei Lu, Min-Yen Kan, and Tat-Seng Chua. 2020. Re-examining the Role of Schema Linking in Text-to-SQL. In *EMNLP*. 6943–6954.

[23] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[24] Boyan Li, Yuyu Luo, Chengliang Chai, Guoliang Li, and Nan Tang. 2024. The dawn of natural language to sql: Are we fully ready? *Proc. VLDB Endow.* 17, 11 (July 2024), 3318–3331.

[25] Boyan Li, Jiayi Zhang, Ju Fan, Yanwei Xu, Chong Chen, Nan Tang, and Yuyu Luo. 2025. Alpha-sql: Zero-shot text-to-sql using monte carlo tree search. *arXiv preprint arXiv:2502.17248* (2025).

[26] Fei Li and Hosagrahar V Jagadish. 2014. Constructing an interactive natural language interface for relational databases. *PVLDB* 8, 1 (2014), 73–84.

[27] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2023. Resdsql: Decoupling schema linking and skeleton parsing for text-to-sql. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 13067–13075.

[28] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. 2024. Codes: Towards building open-source language models for text-to-sql. *PACMMOD* 2, 3 (2024), 1–28.

[29] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. 2024. Can llm already serve as a database interface? a Modeling Ambiguityse grounded text-to-sqls. *NeurIPS* 36 (2024), 42330–42357.

[30] Sheng-Chieh Lin, Akari Asai, Minghan Li, Barlas Oguz, Jimmy Lin, Yashar Mehdad, Wen-tau Yih, and Xilun Chen. 2023. How to train your dragon: Diverse augmentation towards generalizable dense retrieval. *arXiv preprint arXiv:2302.07452* (2023).

[31] Aiwei Liu, Xuming Hu, Lijie Wen, and Philip S Yu. 2023. A comprehensive evaluation of ChatGPT's zero-shot Text-to-SQL capability. *arXiv preprint arXiv:2303.13547* (2023).

[32] Xinyu Liu, Shuyu Shen, Boyan Li, Peixian Ma, Runzhi Jiang, Yuxin Zhang, Ju Fan, Guoliang Li, Nan Tang, and Yuyu Luo. 2025. A survey of text-to-sql in the era of llms: Where are we, and where are we going? *IEEE Transactions on Knowledge and Data Engineering* (2025).

[33] Yuyu Luo, Guoliang Li, Ju Fan, Chengliang Chai, and Nan Tang. 2025. Natural language to sql: State of the art and open problems. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5466–5471.

[34] Ana-Maria Popescu, Oren Etzioni, and Henry Kautz. 2003. Towards a theory of natural language interfaces to databases. In *IUI*. 149–157.

[35] Mohammadreza Pourreza, Hailong Li, Ruoxi Sun, Yeounoh Chung, Shayan Talaei, Gaurav Tarlok Kakkar, Yu Gan, Amin Saberi, Fatma Ozcan, and Sercan O Arik. 2024. Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql. *arXiv preprint arXiv:2410.01943* (2024).

[36] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *NeurIPS*. 36339–36348.

[37] Mohammadreza Pourreza and Davood Rafiei. 2024. Dts-sql: Decomposed text-to-sql with small large language models. *arXiv preprint arXiv:2402.01117* (2024).

[38] Jiexing Qi, Jingyao Tang, Ziwei He, Xiangpeng Wan, Yu Cheng, Chenghu Zhou, Xinbing Wang, Quanshi Zhang, and Zhouhan Lin. 2022. RASAT: Integrating Relational Structures into Pretrained Seq2Seq Model for Text-to-SQL. In *EMNLP*. 3215–3229.

[39] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.

[40] Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. 2022. Evaluating the text-to-sql capabilities of large language models. *arXiv preprint arXiv:2204.00498* (2022).

[41] Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084* (2019).

[42] Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. 2025. OpenAI GPT-5 System Card. *arXiv preprint arXiv:2601.03267* (2025).

[43] Shayan Talaei, Mohammadreza Pourreza, Yu-Chen Chang, Azalia Mirhoseini, and Amin Saberi. 2024. CHESS: Contextual Harnessing for Efficient SQL Synthesis. *arXiv preprint arXiv:2405.16755* (2024).

[44] Lappoon R Tang and Raymond J Mooney. 2001. Using multiple clause constructors in inductive logic programming for semantic parsing. In *ECML*. 466–477.

[45] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).

[46] Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, et al. 2024. Mac-sql: A multi-agent collaborative framework for text-to-sql. *arXiv preprint arXiv:2312.11242* (2024).

[47] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *ACL*. 7567–7578.

[48] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533* (2022).

[49] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *NeurIPS* 33 (2020), 5776–5788.

[50] Ziyang Wang, Yuanlei Zheng, Zhenbiao Cao, Xiaojin Zhang, Zhongyu Wei, Pei Fu, Zhenbo Luo, Wei Chen, and Xiang Bai. 2025. AutoLink: Autonomous Schema Exploration and Expansion for Scalable Schema Linking in Text-to-SQL at Scale. *arXiv preprint arXiv:2511.17190* (2025).

[51] Peng Xu, Wei Ping, Xianchao Wu, Lawrence McAfee, Chen Zhu, Zihan Liu, Sandeep Subramanian, Evelina Bakhturina, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Retrieval meets long context large language models. *arXiv preprint arXiv:2310.03025* (2023).

[52] Navid Yaghmazadeh, Yuepeng Wang, Isil Dillig, and Thomas Dillig. 2017. Sqlizer: query synthesis from natural language. *PACMPL* 1 (2017), 1–26.

[53] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *EMNLP*. 3911–3921.

[54] Yusen Zhang, Ruoxi Sun, Yanfei Chen, Tomas Pfister, Rui Zhang, and Sercan Arik. 2024. Chain of agents: Large language models collaborating on long-context tasks. *Advances in Neural Information Processing Systems* 37 (2024), 132208–132237.

[55] Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv preprint arXiv:1709.00103* (2017).